

Exercise 6: Cursors

Aim

To use MySQL cursors to process records one by one for generating customer statements, applying annual maintenance fees, and updating loan interest rates.

--Utilizing the **same database** from exercise 1, 3, 4 and 5

--**Question:** Write a PL/SQL block using an explicit cursor **GenerateMonthlyStatements** that retrieves all transactions for the current month and prints a statement for each customer.

```
INSERT INTO Transactions VALUES
```

```
(101,1,'Deposit',5000),
```

```
(102,2,'Withdrawal',3000),
```

```
(103,3,'Deposit',7000);
```

```
DELIMITER $$
```

```
CREATE PROCEDURE GenerateMonthlyStatements()
```

```
BEGIN
```

```
DECLARE Done BOOLEAN DEFAULT FALSE;
```

```
DECLARE T_ID INT;
```

```
DECLARE A_ID INT;
```

```
DECLARE T_Type VARCHAR(20);
```

```
DECLARE T_Amount DECIMAL(10,2);
```

```
DECLARE cur CURSOR FOR
```

```
SELECT TransactionID,  
       AccountID,  
       TransactionType,  
       Amount  
FROM Transactions;
```

```
DECLARE CONTINUE HANDLER  
FOR NOT FOUND  
SET Done = TRUE;
```

```
OPEN cur;
```

```
read_loop: LOOP
```

```
FETCH cur  
INTO T_ID,  
     A_ID,  
     T_Type,  
     T_Amount;
```

```
IF Done THEN  
LEAVE read_loop;  
END IF;
```

```
SELECT  
T_ID AS TransactionID,  
A_ID AS AccountID,  
T_Type AS TransactionType,  
T_Amount AS Amount;
```

END LOOP;

CLOSE cur;

END \$\$

DELIMITER ;

CALL GenerateMonthlyStatements();

	TransactionID	AccountID	TransactionType	Amount
►	1	1	Deposit	5000.00

Result 12 × Result 13 Result 14 Result 15 Result 16

	TransactionID	AccountID	TransactionType	Amount
►	4	1	Withdrawal	5000.00

Result 12 Result 13 × Result 14 Result 15 Result 16

	TransactionID	AccountID	TransactionType	Amount
▶	101	1	Deposit	5000.00

Result 12 Result 13 **Result 14** × Result 15 Result 16

	TransactionID	AccountID	TransactionType	Amount
▶	102	2	Withdrawal	3000.00

Result 12 Result 13 Result 14 **Result 15** × Result 16

	TransactionID	AccountID	TransactionType	Amount
▶	103	3	Deposit	7000.00

Result 12 Result 13 Result 14 Result 15 **Result 16** ×

SELECT * FROM transaction;

	TransactionID	AccountID	TransactionType	Amount
▶	1	1	Deposit	5000.00
	4	1	Withdrawal	5000.00
	101	1	Deposit	5000.00
	102	2	Withdrawal	3000.00
	103	3	Deposit	7000.00
*	NULL	NULL	NULL	NULL

transactions 17 ×

--**Question:** Write a PL/SQL block using an explicit cursor **ApplyAnnualFee** that deducts an annual maintenance fee from the balance of all accounts.

DELIMITER \$\$

CREATE PROCEDURE ApplyAnnualFee()

BEGIN

DECLARE Done BOOLEAN DEFAULT FALSE;

DECLARE A_ID INT;

DECLARE cur CURSOR FOR

SELECT AccountID

FROM Account;

DECLARE CONTINUE HANDLER

FOR NOT FOUND

SET Done = TRUE;

OPEN cur;

read_loop: LOOP

FETCH cur

INTO A_ID;

IF Done THEN

LEAVE read_loop;

END IF;

UPDATE Account

SET Balance = Balance - 100

WHERE AccountID = A_ID;

END LOOP;

CLOSE cur;

END \$\$

DELIMITER ;

```
SELECT * FROM Account;
```

	AccountID	CustomerID	AccountType	Balance
▶	1	101	Savings	45400.00
	2	102	Savings	35200.00
	3	103	Savings	15050.00
	4	104	Current	40300.00
	5	105	Savings	25150.00
*	NULL	NULL	NULL	NULL

```
CALL ApplyAnnualFee();
```

```
SELECT * FROM Account;
```

	AccountID	CustomerID	AccountType	Balance
▶	1	101	Savings	45300.00
	2	102	Savings	35100.00
	3	103	Savings	14950.00
	4	104	Current	40200.00
	5	105	Savings	25050.00
*	NULL	NULL	NULL	NULL

--**Question:** Write a PL/SQL block using an explicit cursor **UpdateLoanInterestRates** that fetches all loans and updates their interest rates based on the new policy.

```
DELIMITER $$
```

```
CREATE PROCEDURE UpdateLoanInterestRates()
```

```
BEGIN
```

```
DECLARE Done BOOLEAN DEFAULT FALSE;
```

```
DECLARE C_ID INT;
```

```
DECLARE cur CURSOR FOR
```

```
SELECT CustomerID  
FROM Customer;
```

```
DECLARE CONTINUE HANDLER  
FOR NOT FOUND  
SET Done = TRUE;
```

```
OPEN cur;
```

```
read_loop: LOOP
```

```
FETCH cur  
INTO C_ID;
```

```
IF Done THEN  
LEAVE read_loop;  
END IF;
```

```
UPDATE Customer
```

```
SET LoanInterestRate = LoanInterestRate - 0.5
```

```
WHERE CustomerID = C_ID;
```

```
END LOOP;
```

```
CLOSE cur;
```

```
END $$
```

```
DELIMITER ;
```

```
SELECT CustomerID, CustomerName, LoanInterestRate FROM Customer;
```

	CustomerID	CustomerName	LoanInterestRate
▶	101	Krishna	2.50
	102	Rahul	10.00
	103	Priya	1.50
	104	Arun	11.00
	105	Sneha	2.00
	106	Karan	9.50
*	NULL	NULL	NULL

CALL UpdateLoanInterestRates();

SELECT CustomerID, CustomerName, LoanInterestRate FROM Customer;

	CustomerID	CustomerName	LoanInterestRate
▶	101	Krishna	2.00
	102	Rahul	9.50
	103	Priya	1.00
	104	Arun	10.50
	105	Sneha	1.50
	106	Karan	9.00
★	NULL	NULL	NULL